

Constructing Test Cases from Coverage Requirements by Symbolic Evaluation

Karl Meinke¹

School of Electrical Engineering and Computer Science,
KTH Royal Institute of Technology, Stockholm, Sweden.

Constructing Test Cases from Coverage Requirements

We consider the problem of constructing a test case tc that satisfies a given test coverage requirement tr . For some reason, this subject is frequently omitted in introductory texts to software testing. Perhaps there is an assumption that the problem is relatively straightforward and easy to solve. For simple programs this can be true. However, in general, test case construction is a difficult, error prone and time-consuming task if it must be done manually. As we have already mentioned, on theoretical grounds there is no general algorithm for test case construction that solves the problem in every situation. This is due to the algorithmic undecidability of a great many constraint solving problems, starting with the famous Halting Problem (see (Sipser, 2012)). Even those constraint solving problems which are algorithmically solvable (such as the Boolean SAT problem), often have a prohibitive time complexity for large problem instances.

We will present a systematic approach to test case construction from a given path requirement tr by *symbolic evaluation* of the path. This means we combine the code information found along the path (assignments and Boolean conditions) into a single Boolean constraint that characterizes the test cases satisfying that path requirement. By learning a simple set of rules for assignments and conditions, and by combining these rules with common algebraic laws about programming language predicates and operators (such as equations, inequalities and the laws of Boolean algebra) it is possible to rigorously derive valid test case constraints. The goal is to correctly derive a constraint C , in such a way that:

- (i) we can assert with confidence that a test coverage requirement tr is *satisfiable* by some test case tc , and can correctly construct a satisfying test case tc , either manually or with the help of an algorithm or tool.
- (ii) we can detect if a test requirement tr is *unsatisfiable*. In this case, there is typically a code component that is unreachable or “dead code”.

We will illustrate the process of symbolic evaluation to derive a test case constraint from a path requirement by means of a simple concrete example. Hopefully this

¹ Copyright: Karl Meinke 2025

example is sufficiently generic that the reader can apply the same principles to other examples.

Consider the following simple code example.

```
x = 2*x;
y = x+y;

while (y+x > 100) {
    x = x + 1;
    if (x mod 2 == 0) { y = y-1; } else { y = y-2; }
};
return y;
```

The condensation graph for this code is given in Figure 1 Condensation graph.

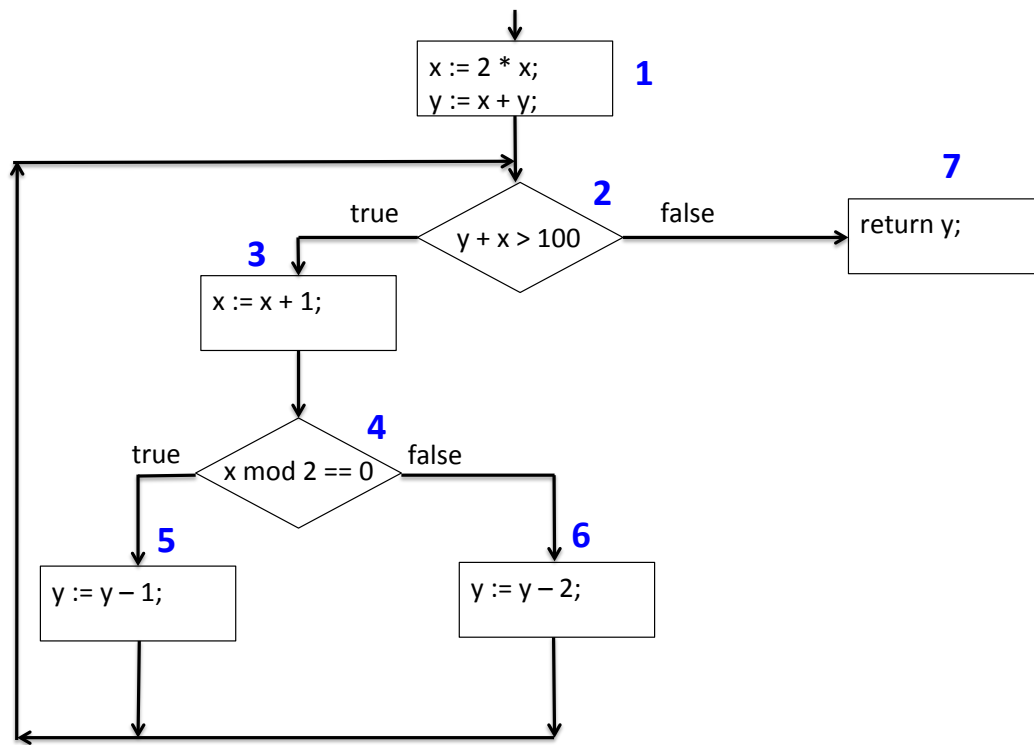


Figure 1 Condensation graph

This graph clearly shows typical CFG features such as a loop and a branch.

Consider the concrete test requirement tr which is the path

$tr = 1, 2, 3, 4, 6, 2, 7$

This path tr traverses the loop body exactly once.

In general, for any test requirement $tr = v_1, \dots, v_k$, we wish to be able to find a test case tc , (i.e. an assignment of values to the SUT input variables) that traverses *all* of the nodes $v_1, \dots, v_k$. We would really like a general method that solves the problem of finding tc for any condensation graph and any path requirement.

Our approach will be to derive a logical constraint $C(x_1, \dots, x_n)$ on the input variables $x_1, \dots, x_n$ of the SUT (for the above SUT these are x, y) such that:

- (i) every solutionⁱ α of $C(x_1, \dots, x_n)$ leads SUT execution down the path tr , and
- (ii) if α is a test case that leads execution down the path tr then α is a solution of the constraint $C(x_1, \dots, x_n)$.

So $C(x_1, \dots, x_n)$ will exactly characterize all the test cases satisfying tr , and if the set of solutions of $C(x_1, \dots, x_n)$ is empty, i.e. C is unsatisfiable then we can be certain that tr is a dead or unreachable path.

What makes the problem of constructing $C(x_1, \dots, x_n)$ difficult is that although the Boolean conditions of the original code are the key to finding C , the presence of assignment statements (such as those in node 1 above) has a modifying effect on the actual code conditions. To combine the relevant assignments and Boolean conditions of tr into a single constraint, we must traverse tr as a path from start to finish, and collect and integrate the assignments and Boolean conditions along the way. Often it is very useful to simplify the constraint C using common mathematical laws, such as the laws of Boolean algebra, after each path step. In this way we may more easily find test cases as solutions.

In Table 1 Constraint table for the path $tr = 1,2,3,4,6,2,7$ below we present a *constraint table* for the path $tr = 1,2,3,4,6,2,7$.

This table consists of one line for each code instruction along the path. We number the associated graph node of the instruction in the left-hand column. We break each assignment block down into its individual assignment statements, to make each step in the construction of C smaller and simpler. In the middle column, we show the actual code instruction along the path tr . In the right-hand column of the table, we show the *current constraint* arising from either:

- (i) executing the assignment on the current line, or
- (ii) evaluating the Boolean condition on the current line to either *true* or *false*, depending on which next node we progress to.

Notice in the constraint column that each code variable can appear as itself, e.g. x or in primed form e.g. x' . The unprimed variable e.g. x stands for the value of that variable at the *start of code execution*. An unprimed variable is a variable we must ultimately find a value for, in order to build a test case. The primed variable e.g. x'

stands for the value of x *after execution of the instruction* in the middle column. So, the constraint on each line expresses a relationship between the values of program variables *initially* and their values *currently* on each line.

For example, in line 1, the code instruction in column 2 from node 1 is $x := 2 * x$. So not surprisingly the relationship between x before this instruction and x after this instruction (i.e. x') in column 3 is:

$$x' = 2 * x$$

Line 2 has a second assignment instruction $y := x + y$ from node 1. To understand its effect on the constraint C in column 3, we must replace the variable x in the rhs expression $x + y$ by its current value immediately before the assignment, which is x' . Now according to line 1, $x' = 2 * x$. Substituting x with $2 * x$ in $x + y$ gives

$$y' = 2 * x + y.$$

In line 3 we encounter the loop guard $y + x > 100$. In order to proceed to node 3 we must make this Boolean condition *true*. Now $y + x > 100$ will be true precisely when the current value of x plus the current value of y is greater than **100**. From lines 1 and 2 we can see these current values x' and y' . They are again substituted into the code line $y + x > 100$ giving the constraint $((2 * x) + y) + 2 * x > 100$ on line 3 in the right hand column. In this way, making assignment substitutions and condition substitutions using the current values of variables from previous lines, we can derive a constraint for every code line.

In particular we eventually see that to arrive at node 2 for the second time we must satisfy the constraint

$$4x + y = 101$$

This shows that the only test cases which lead down *tr* are assignments $\alpha(x)$, $\alpha(y)$ for x and y which satisfy the linear equation $4\alpha(x) + \alpha(y) = 101$. For example, let $\alpha(x) = 25$ and $\alpha(y) = 1$ then the constraint is satisfied. Manually tracing the code execution for these input values should confirm our calculations.

.

Graph Node	Instruction	Constraint
1	$x := 2 * x$	$x' = 2 * x$
1	$y := x + y$	$y' = 2 * x + y$
2	$y + x > 100$	$((2 * x) + y) + 2 * x > 100$
3	$x := x + 1$	$x' = 2 * x + 1$
4	$x \bmod 2 \neq 0$	$((2 * x) + y) + 2 * x > 100 \ \& \ (2 * x + 1 \bmod 2 \neq 0)$ $\equiv$ $((2 * x) + y) + 2 * x > 100 \ \& \ \text{TRUE}$ $\equiv$

		$((2*x)+y) + 2*x > 100$
6	$y := y - 2$	$y' = (2*x + y) - 2$
2	$y + x \leq 100$	$((2*x + y) - 2) + (2*x + 1) \leq 100 \ \&$ $((2*x)+y) + 2*x > 100 \ \&$ $(2*x + 1 \bmod 2 \neq 0)$ $\equiv$ $((2*x + y) - 2) + (2*x + 1) \leq 100 \ \&$ $((2*x)+y) + 2*x > 100 \ \&$ TRUE $\equiv$ $4x + y > 100 \ \& \ 4x + y \leq 101$ $\equiv$ $4x + y = 101$
7	return y	$4x + y = 101$

Table 1 Constraint table for the path $tr = 1,2,3,4,6,2,7$

The constraint derived on test cases is by no means always as simple as a linear equation (which always has a solution). Table 2 constraint table for the path $tr = 1,2,3,4,5,2,7$ gives a similar analysis for the path $tr = 1,2,3,4,5,2,7$ in the same condensation graph. This time we see from line 5 that the constraint $C(x,y)$ is always false, i.e. unsatisfiable. This means there is no test case that leads to execution of tr . The code at node 5 is therefore dead or unreachable code.

Graph Node	Instruction	Constraint
1	$x := 2 * x$	$x' = 2 * x$
1	$y := x + y$	$y' = 2*x + y$
2	$y + x > 100$	$((2*x)+y) + 2*x > 100$
3	$x := x + 1$	$x' = 2*x + 1$
4	$x \bmod 2 == 0$	$((2*x)+y) + 2*x > 100 \ \&$ $(2*x + 1 \bmod 2 = 0)$ $\equiv$ $((2*x)+y) + 2*x > 100 \ \& \ \text{FALSE}$ $\equiv$ FALSE
5	$y := y - 1$	Unreachable along this path
2	$y + x \leq 100$	Unreachable along this path
7	return y	Unreachable along this path

Table 2 constraint table for the path $tr = 1,2,3,4,5,2,7$

Acknowledgements

The author gratefully acknowledges financial support from the EU ITEA3 TESTOMAT Project 16032.

Bibliography

- Ammann, P., & Offutt, J. (2017). *Introduction to Software Testing* (2nd uppl.). Cambridge University Press.
- Myers, G. (1979). *The Art of Software Testing* (1st uppl.). Wiley.
- Lyu, M. (1996). *Handbook of Software Reliability Engineering*. USA: McGraw-Hill, Inc.
- Biggs, N. (2003). *Discrete Mathematics*. Oxford University Press.
- Sipser, M. (2012). *Introduction to the Theory of Computation*. Course Technology.
- Meinke, K., Bennaceur, A., & Hähnle, R. (2018). *Machine Learning for Dynamic Software Analysis: Potentials and Limits*. Springer.
- Wirth, N. (1976). *Algorithms + Data Structures = Programs*. Prentice Hall.

ⁱ The notation $C(x_1, \dots, x_n)$ means that C is a logical formula in which only the variables $x_1, \dots, x_n$ appear. A solution of $C(x_1, \dots, x_n)$ is an assignment of values to $x_1, \dots, x_n$ that makes $C(x_1, \dots, x_n)$ true.